SE3040 – Application Frameworks
BSc (Hons) in Information Technology Specialized in Software Engineering
Year 03

Assignment Specification - 2026

Full Stack Application Development

Overview:

This assignment involves developing a full-stack software application. Students will work in groups of four to design and implement the system. The project will be completed in two milestones, allowing progress to be reviewed and improved in stages.

Part 1. A Secure RESTful API Backend

You will build a backend system based on the ***specific topic assigned*** to your group. Each group will receive a unique domain (e.g., event management, healthcare system, inventory system, student management, travel app backend, etc.).

Backend Requirements:

a. Develop a full RESTful API using Express.js (Node.js)
b. Clearly define the functional requirements of each component (a project should have at least 4 components)
c. The backend must include individual components, each of which must fulfill the following requirements.
    i. All data manipulation and business logic must be exposed through properly structured REST endpoints following standard HTTP methods and status codes (CRUD operations)
    ii. Additional Feature (use of third-party API)
    iii. MongoDB database integration
    iv. Protected routes & role-based access
    v. Validation and Error Handling
    vi. Clean architecture and best practice
    vii. API documentation (Swagger/Postman)

Part 2. A React Frontend Application

In addition to the backend, each group must develop a React frontend application for their backend. The React app must consume the backend API developed in Part A and serve as the user interface for the full-stack system. Students must also demonstrate how state management, session handling, and deployment were handled in the project.

Classification: Public-SLIIT

Frontend Requirements:

a. React functional component architecture (React Hooks, Context API or Redux for state management)
b. API Integration
   The frontend must consume the backends' endpoints, such as (user management, Entity management – CRUD, Filters, Pagination, search etc.)
c. UI/ UX Requirements (Tailwind CSS, Bootstrap etc.)
d. Session Management
e. Deployment

Deployment Requirements:

Backend Deployment: The RESTful API developed using **Express.js (Node.js)** must be deployed on a cloud hosting platform such as Render, Railway, OR any other platform.

Frontend Deployment: The React frontend application must be deployed using platforms such as Vercel, Netlify, Firebase Hosting, OR any other platform.

Deployment Documentation: Students must include a Deployment section in the README.md file containing:

- Backend deployment platform and setup steps
- Frontend deployment platform and setup steps
- Environment variables used (without exposing secrets)
- Live URLs for:
  - Deployed backend API
  - Deployed frontend application
- Screenshots or evidence of successful deployment.

Additional Testing Requirements:

1. Unit Testing:

- Implement unit tests for individual components and functions to validate their behavior in isolation.

2. Integration Testing:

- Conduct integration tests to ensure different parts of the application work together seamlessly. This includes testing interactions between controllers, services, and the MongoDB database.
- Test API endpoints to verify correct handling of requests and responses, including error scenarios.

3. Performance Testing:

- Evaluate the API's performance under various loads to ensure it can handle multiple requests simultaneously without significant latency.
- Tools like Artillery.io (for Express JS applications) can be used for performance testing.

Evaluation Criteria:

- Correctness and functionality of the application.
- Creativity and design implementation.
- Code quality, including readability and use of best practices.
- Completeness of documentation.
- Regularity and informativeness of git commit.
- Thoroughness of testing.
- Viva performance.

Submission Guidelines:

1. Source Code Submission
    - Submit your source code through **GitHub**
    - Proper **git workflow** will be evaluated from the beginning of the work. (Please note that any amendments made beyond the specified deadline will automatically incur zero marks. Compliance with submission timelines will be assessed during the viva)
    - Maintain clear commit history and meaningful commit messages.
2. Documentations
    - Include a comprehensive **README** file containing:
        i. Setup Instructions: Step-by-step guide to get the project running

Classification: Public-SLIIT

ii. API Endpoint Documentation: Complete documentation of all API endpoints, including: HTTP methods, Request/response formats, Authentication requirements, Example requests and responses

- Include the Deployment Report.
- Include the Testing Instruction Report:
  i. How to run unit tests
  ii. Integration testing setup and execution
  iii. Performance testing setup and execution
  iv. Testing environment configuration details

3. Deadline:
   - Evaluation 01: 27th February 2026
   - Evaluation 02: 12th April 2026

Marking Rubric – Evaluation 01 (100%)

Students are mandated to present evidence of **80% of backend development completion at evaluation 01**. During the evaluation students are required to deliver a 10 minutes presentation, followed by a 5 minutes demonstration. The presentation shall systematically document the developmental progress and incorporate the requisite components outlined below:

Description	Excellent	Good	Average	Below Average
Group Contribution (20%)				
Presentation – Project Planning – Architecture – Justification for the selected framework and progress completion (20 marks)	Comprehensive system architecture with clear component definitions, database schema, API design, and technology stack justification (20 marks)	Good architecture with most components defined and reasonable design decisions (15 marks)	Basic architecture with minimal planning, unclear component separation (10 marks)	No clear architecture or planning demonstrated (5 marks)
Individual Contribution (80%)				
Core Functionalities – backend (40 marks)	Component functional with at least 4 API endpoints working (40 marks)	Component with basic CRUD operations partially implemented (30 marks)	Component setup initiated with 1-2 endpoints working (20 marks)	No clear backend progress (5 marks)
MongoDB integration established (20 marks)	MongoDB integration established (20 marks)	Database connected and schema defined (15 marks)	Basic database setup (10 mark)	No or incomplete setup done (2 marks)
Clean code structure and following best practices (10 marks)	Code is very well-structured, and consistently follows best practices (10 marks)	Code is well-organized and mostly follows best practices with minor inconsistencies (8 marks)	Code structure is acceptable but inconsistent. Follow some best practices (5 marks)	Code is not poorly structured and does not follow best practices. (1 mark)
Presentation and Communication Skills (10 marks)	Clear, confident, and well-structured presentation with logical flow (10 marks)	Organized presentation with generally clear explanations (8 marks)	Basic presentation structure with some unclear or disorganized sections (5 marks)	Poorly organized or unclear presentation (1 mark)

Classification: Public-SLIIT

Marking Rubric – Evaluation 02 - Final (100%)

During the evaluation students are required to prepare for a 10 minutes complete demonstration, followed by a 10 minutes Q&A session.

Description	Excellent	Good	Average	Below Average
Group Contribution (30%)				
Component Design & Business Logic (10 marks)	Well-defined component with clear business requirements. Proper service layer architecture (10 marks)	Good component design with minor architectural issues (8 marks)	Basic component with acceptable business logic implementation (5 marks)	Poorly structured component with business logic issues (2 marks)
Component Architecture & State Management (10 marks)	Well-structured functional components, proper use of hooks, effective state management (Context API/Redux) (10 marks)	Good component structure with minor issues (8 marks)	Basic functional components, minimal state management (5 marks)	Poor component architecture (2 marks)
Documentation and Deployment (10 marks)	Comprehensive README with project overview. Both frontend and backend successfully deployed (10 marks)	Basic documentation with missing details. Both frontend and backend successfully deployed with minor issues (8 marks)	Basic documentation with missing details. Partial deployment or deployment with issues (5 marks)	Poor or no documentation. Not deployed (2 marks)
Individual Contribution (70%)				
RESTful API Development (5 marks)	Complete RESTful API with all CRUD operations (5 marks)	All CRUD operations functional with minor REST convention issues (4 marks)	Most CRUD operations are working, some inconsistencies in API design (3 marks)	No or poor functional API (1 mark)
Database Integration & Data Modeling (10 marks)	Well-designed MongoDB schema with proper field types and efficient data modeling (10 marks)	Functional database with acceptable schema design (8 marks)	Basic database integration with design issues (5 marks)	Poor or no database integration (2 marks)
Additional features & Third-Party Integration (10 marks)	Meaningful third-party integration working properly (10 marks)	Basic third-party integration with limited functionality (8 marks)	Basic third-party integration with inconsistencies in functionality (5 marks)	No additional features (2 marks)
API Integration & Functionality (5 marks)	Complete API integration, all CRUD operations working, proper error handling, loading states (5 marks)	Most API endpoints are integrated with minor issues (8 marks)	Basic API integration with significant gaps (5 mark)	No API integration (2 marks)
Git workflow (5 marks)	Properly initiated and maintained standard git workflow (5 marks)	Good initiation and most of the standards followed (4 marks)	Maintained some of the git workflow standards (3 marks)	No maintenance of git workflow (1 mark)
UI/UX Design & Session Management (10 marks)	Professional, responsive UI using CSS framework. Protected routes implemented (10 marks)	Good UI with functional session management (8 marks)	Basic UI and session handling (5 marks)	Poor UI/UX or no session management (2 marks)
Testing (5 marks)	Comprehensive unit testing, Integration testing and Performance testing (5 marks)	Basic unit testing, Integration testing and Performance testing (4 marks)	Minimal unit testing, Integration testing and Performance testing (3 mark)	Poor unit testing, Integration testing and Performance testing (1 mark)

Classification: Public-SLIIT

Viva Performance (20 marks)	Deep understanding of the project and the implementation (20 marks)	Good understanding with some gaps (15 marks)	Basic understanding, struggles with explanations (8 marks)	Poor understanding (5 marks)

Classification: Public-SLIIT